

SOUNDNESS IS NOT SECURITY

LAYERED VERIFICATION OF BPMN COLLABORATIONS WITH THE TAMARIN PROVER

ADDING AN ADVERSARY TO THE TOKEN GAME

MARTIN FARKAS · IMRE KOCSIS

AVM'26 · ALPINE VERIFICATION MEETING · EGER · 22 TO 24 SEPTEMBER 2026

Based on: Farkas, Kocsis. *Trust No Pool: Formal Security Analysis of BPMN Collaborations*. BPM 2026, foundations track, to appear.



LEARNING OUTCOMES

READ

You can read a Tamarin theory as a Petri net with an adversary: places, tokens, transitions, and the network rules.

BUILD

You can structure a model in two layers, so that changing the channel means replacing one small file rather than rewriting the model.

SPECIFY

You can say what soundness does not check about a collaboration, and write it as a trace lemma the prover decides.

I

A sound collaboration

II

From Petri nets to Tamarin

III

One process,
several channels

IV

Checking the idealization

V

Exercise

Prerequisites: propositional and first-order logic, and the idea of a Petri net. No prior Tamarin, BPMN, or cryptography is assumed. The repository has a 20-minute take-home exercise and a Docker command that runs everything.



Part I

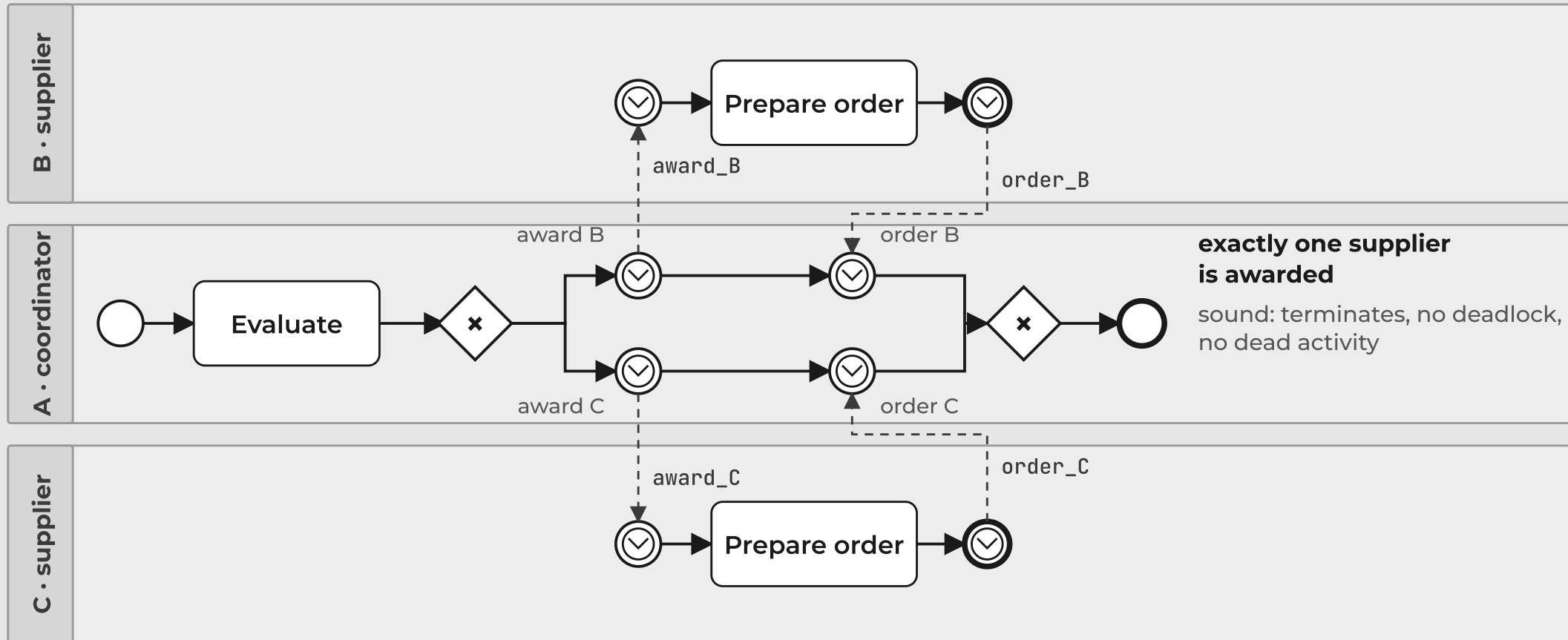
A SOUND COLLABORATION

CORRECTNESS UNDER AN ASSUMPTION OF HONEST PARTICIPANTS

Soundness analysis verifies correctness under the assumption that every participant executes its pool faithfully. Security analysis asks what a dishonest participant can achieve.



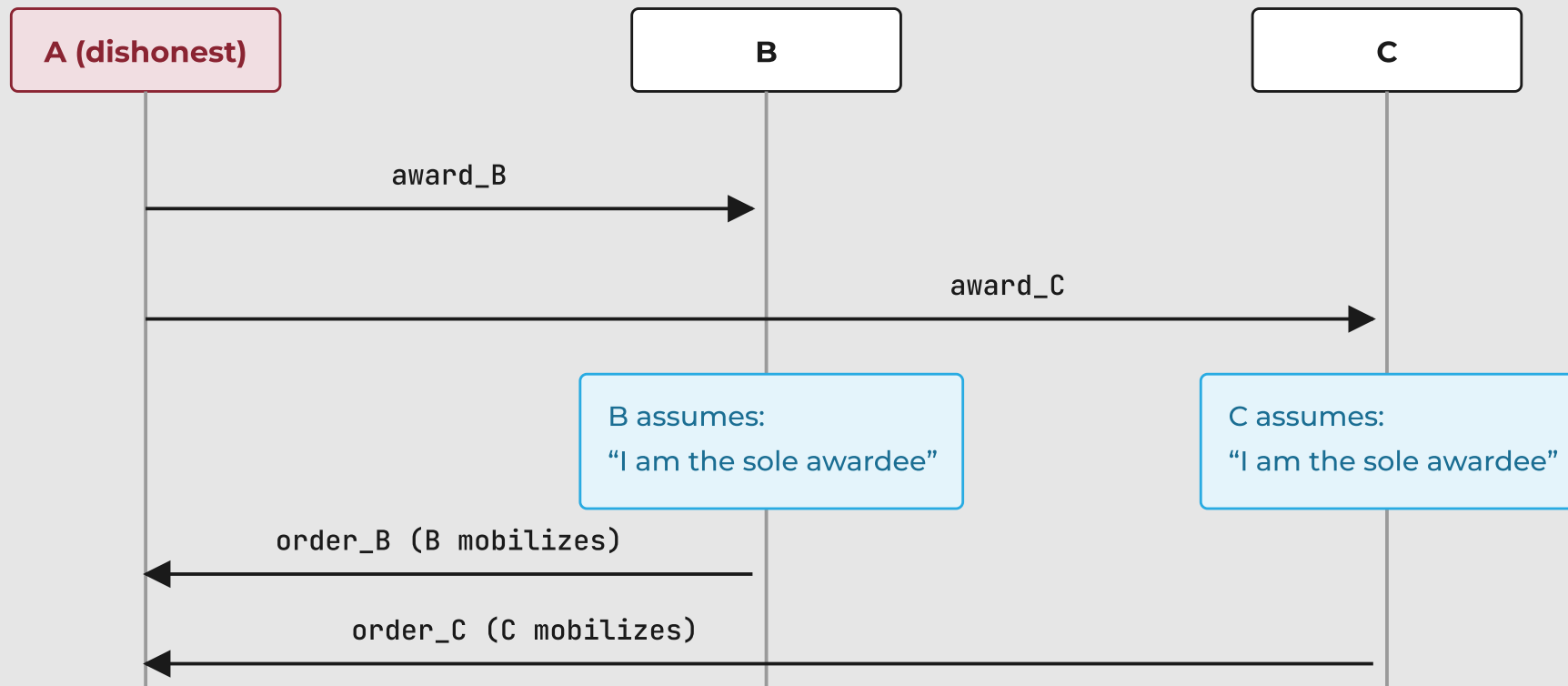
RUNNING EXAMPLE: A PROCUREMENT COLLABORATION



Three pools. A takes one branch of an exclusive gateway; B or C prepares an order. The model is sound: every instance terminates, with no deadlock and no dead task.

EXAMPLE · A DISHONEST COORDINATOR

A DISHONEST COORDINATOR AWARDS BOTH SUPPLIERS

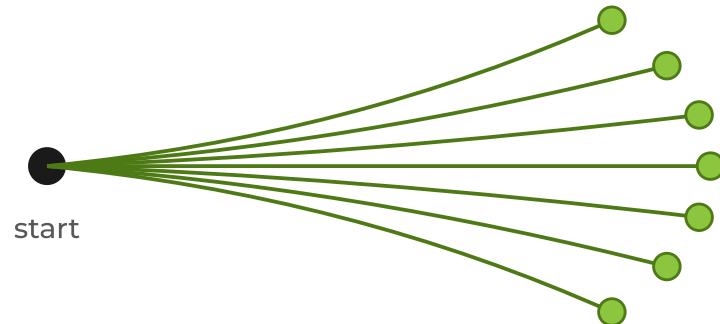


A sends an award to both suppliers. B and C both prepare an order on a false assumption. The model does not forbid this: BPMN's execution model assumes that every pool executes its role faithfully.

SOUNDNESS ANALYSIS VERSUS SECURITY ANALYSIS

Soundness

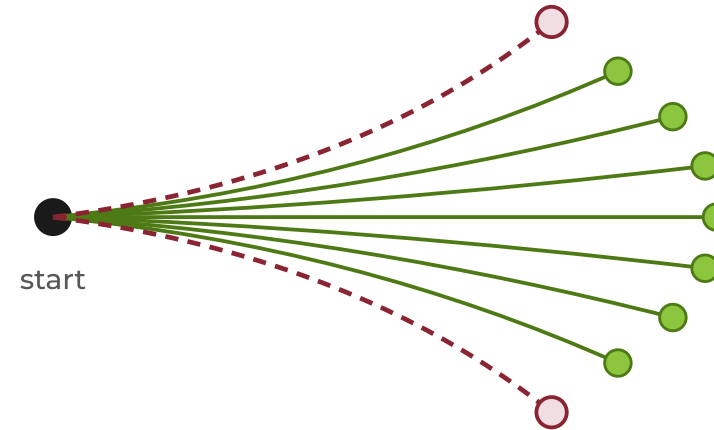
$\forall$ trace of honest participants:
terminates, no deadlock



all traces are honest

Security

$\exists$ trace where one participant deviates
and another acts on a false assumption



B and C both awarded

the deviating participant's traces are outside the honest set

Soundness is a statement about all traces of honest participants. Security is a statement about the existence of a trace in which one participant deviates and another acts on a false assumption.

COLLABORATIONS AS SECURITY PROTOCOLS

PARTICIPANTS

Principals. Each pool runs its own code and may deviate from it.

EXECUTION PLATFORM

The trusted third party, or the absence of one: message bus, smart contract, credentials.

ATTACKER GOALS

The gap between what BPMN assumes (faithful execution, untampered messages) and what the platform enforces.

Security annotations state goals, not capabilities.

SecBPMN and related notations express what you want. They do not express what a dishonest pool *can do* on a given platform, nor do they check whether it is prevented.

A protocol verifier covers every instance and every derivation.

The proof holds for every number of instances and every adversary derivation, the equational theory does the cryptography, and search runs backward from the violation. The adversary is a set of rules already in the theory, not a process you compose in and bound.

This framing, the threat model, and the platform results are from our paper: Farkas, Kocsis. *Trust No Pool: Formal Security Analysis of BPMN Collaborations*. BPM 2026, foundations track, to appear.



Part II

FROM PETRI NETS TO TAMARIN

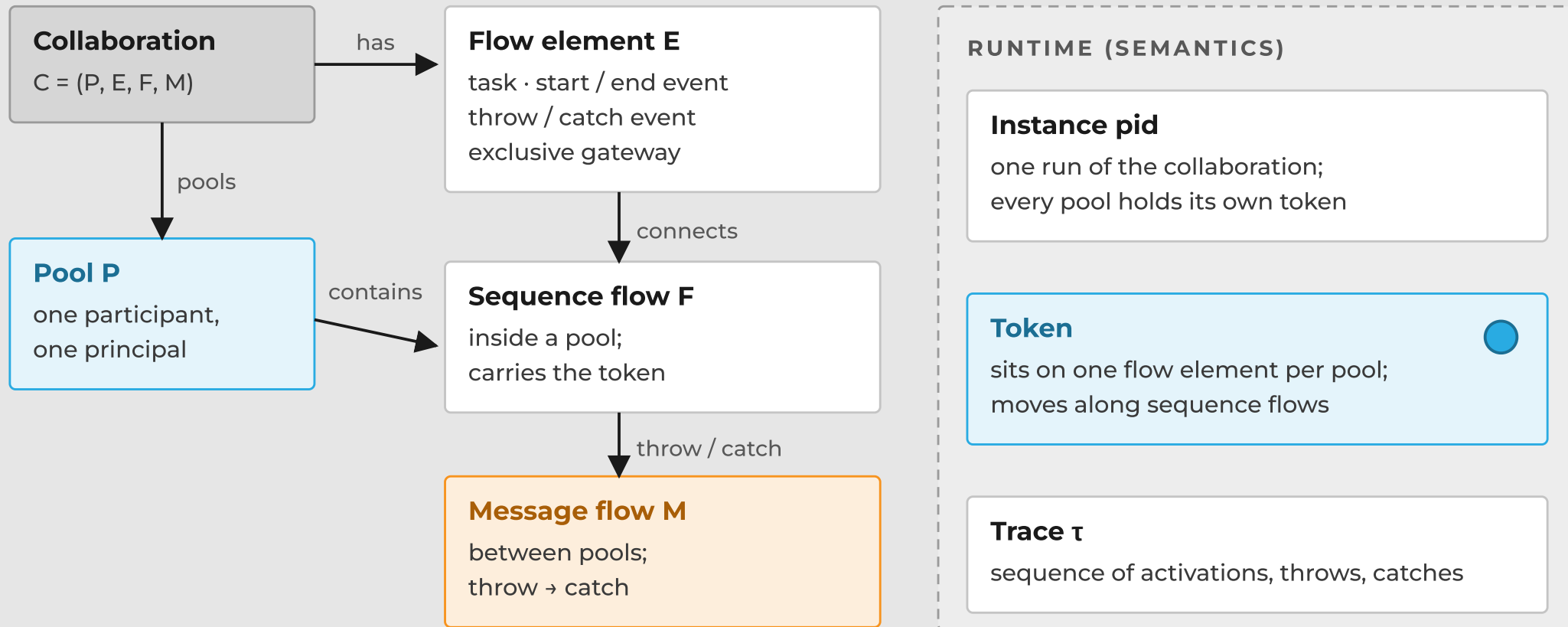
META-MODEL · GLOSSARY · THREE CONSTRUCTS

A pool is a rule set, a token is a linear fact, and
the adversary is a small set of additional rules.



META-MODEL · BPMN SUBSET

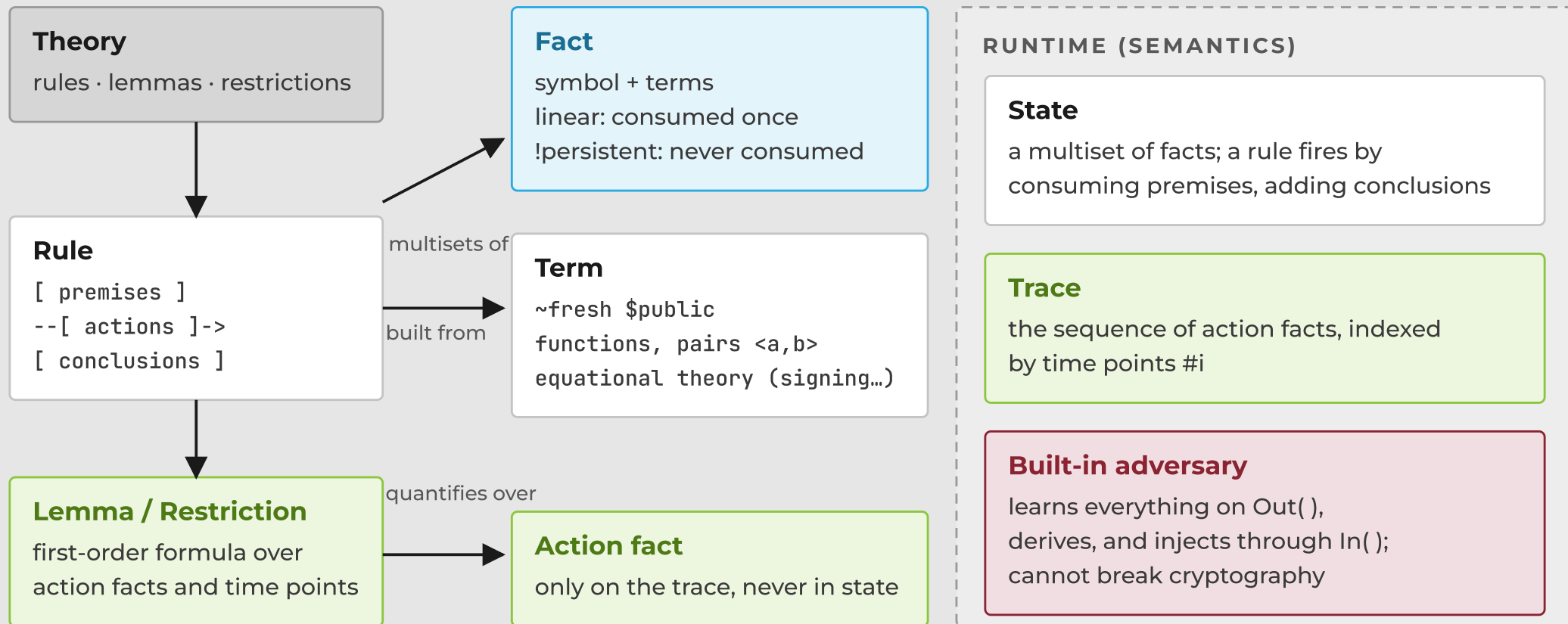
THE BPMN COLLABORATION META-MODEL



Left, the modeling language: pools, flow elements, sequence and message flows. Right, the runtime: instances, one token per pool, traces. The execution model assumes that every pool executes its own lane faithfully.

META-MODEL · TAMARIN THEORIES

THE TAMARIN THEORY META-MODEL



A theory is rules, lemmas, and restrictions. A rule consumes a multiset of facts, records action facts, and produces a multiset of facts. The built-in adversary has no Petri-net counterpart.

GLOSSARY · FOUR NOTATIONS

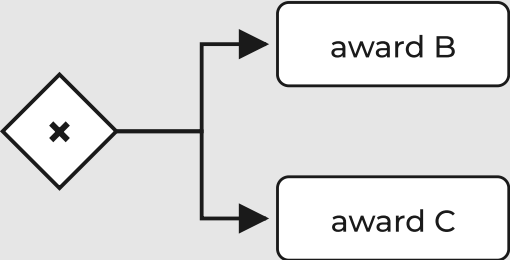
GLOSSARY

Petri net	Multiset rewriting	Tamarin syntax	BPMN collaboration
place	fact symbol	<code>St(pid, 'A', 'gateway')</code>	a flow element a token can sit on
token	linear fact instance	consumed exactly once	the token of a pool in an instance
transition	rewrite rule	<code>[l] --[a]-> [r]</code>	task, event, gateway branch
marking	multiset state	all current facts	state of all pools
firing sequence	trace	<code>Act, Sent, Received</code> at #i	execution
.	persistent fact	<code>!ModelEdge(...)</code> <code>!Auth(...)</code>	the collaboration graph itself
.	fresh name	<code>Fr(~pid)</code>	instance id
.	adversary rules	<code>Out</code> <code>In</code> <code>K</code>	the dishonest pool + the wire
.	trace formula	<code>All ... @ #i ==> Ex ... @ #j</code>	an attacker goal, negated

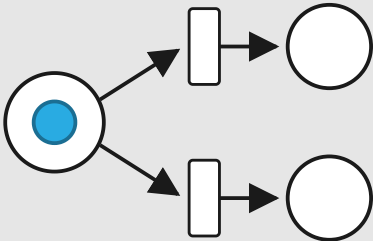


THE EXCLUSIVE GATEWAY IN THREE NOTATIONS

BPMN



PETRI NET



TAMARIN RULES

```
rule A_Gateway_B:
[ St(pid,'A','gateway') ]
--[ Act(pid,'A','gateway','branch_B') ]->
[ St(pid,'A','throw_award_B') ]

rule A_Gateway_C:
[ St(pid,'A','gateway') ] ... 'branch_C' ...
```

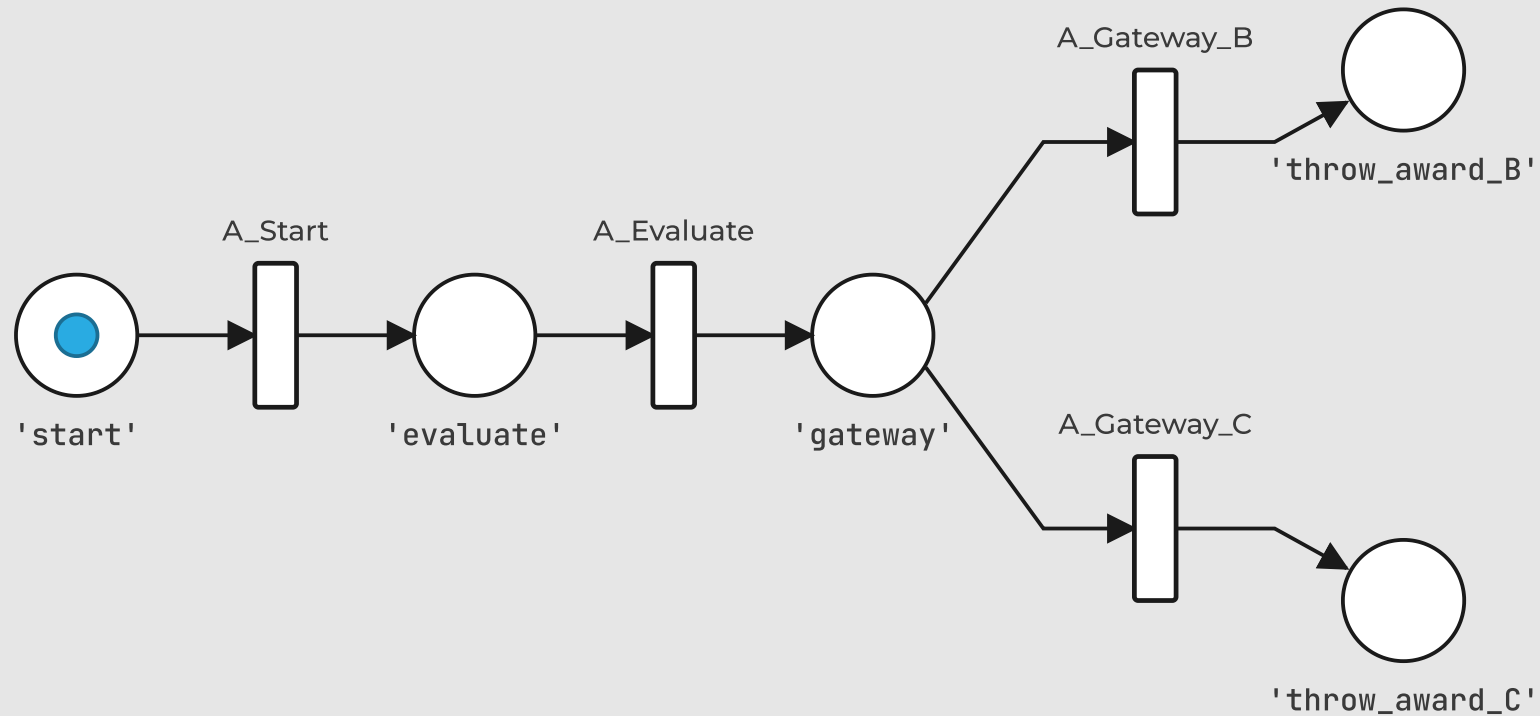
place	token	transition	marking	firing sequence
fact symbol	linear fact instance	rewrite rule	multiset state	trace
St(pid, pool, element)	one St per pool	[l]--[a]->[r]	all current facts	action facts at #i
Cervesato & Scedrov: place/transition nets embed into multiset rewriting. Tamarin adds terms, fresh names, and an adversary on top.				

The exclusive gateway corresponds to two transitions sharing one token, and to two rules sharing one linear premise. Exclusivity follows from consumption of the shared fact; no further constraint is needed.



CONSTRUCT 1 · RULES OVER A MULTISET OF FACTS

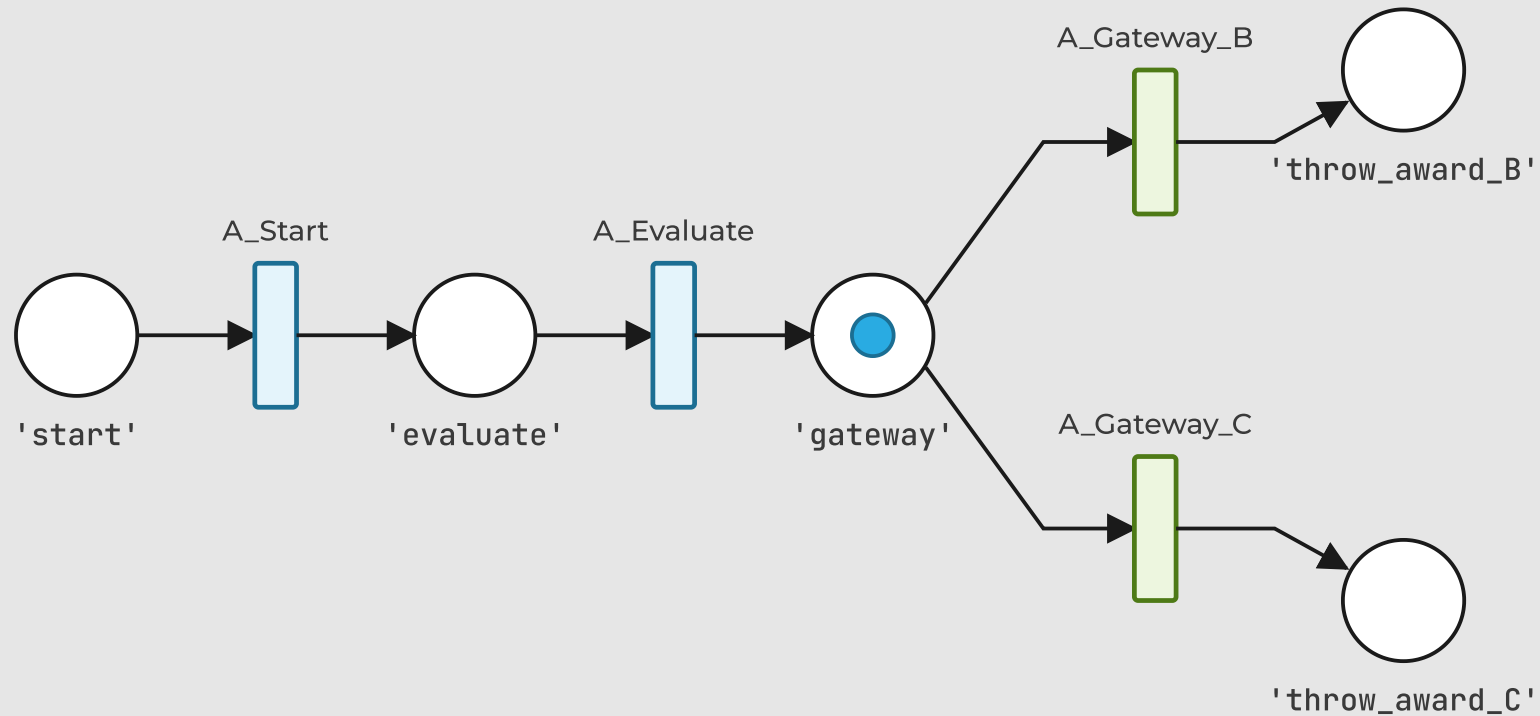
TOKEN GAME: ONE TOKEN PER POOL



Init puts one token in each pool of an instance. A linear fact is a token: it is consumed exactly once, so the element it sits on is where that pool currently is.

CONSTRUCT 1 · RULES OVER A MULTISET OF FACTS

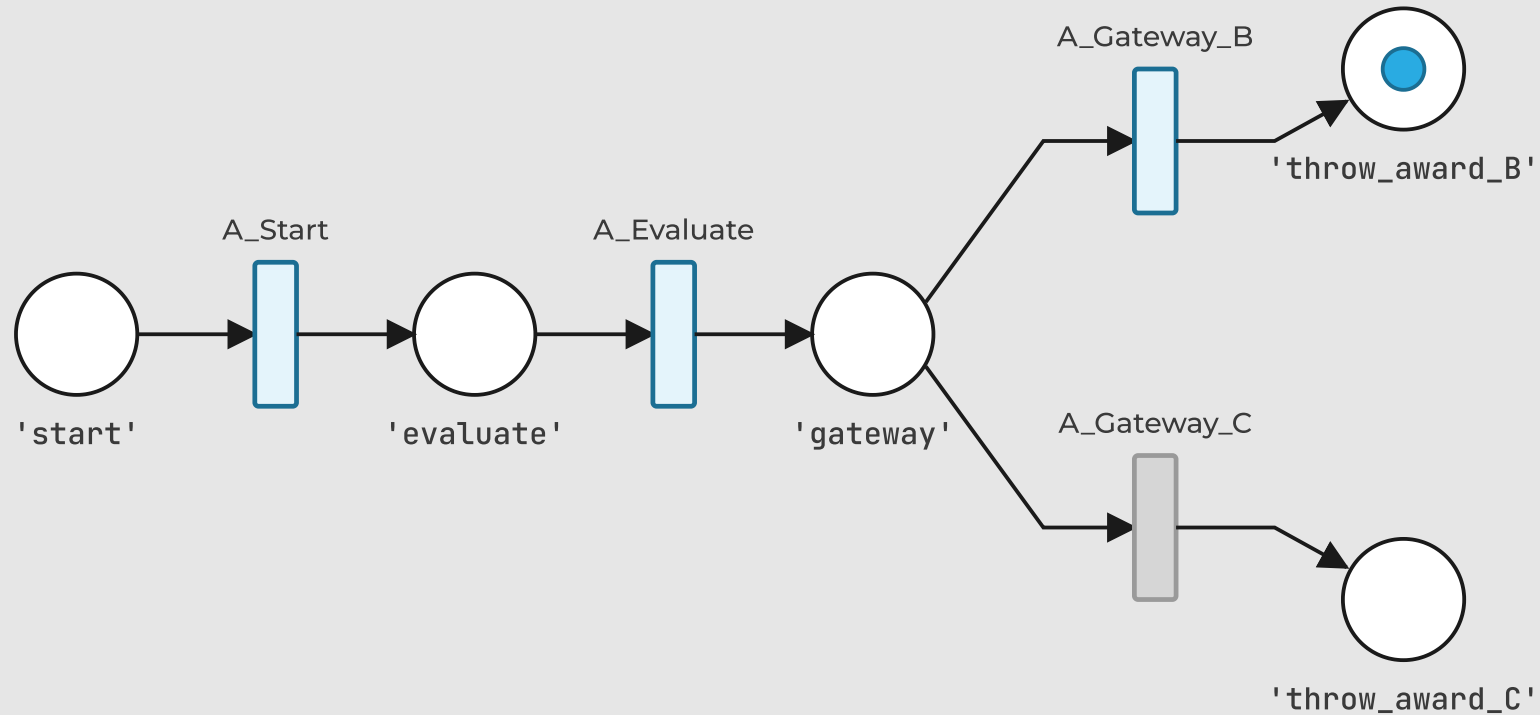
TOKEN GAME: THE GATEWAY IS ENABLED



The token has reached the gateway. Both branch rules are enabled.
At most one fires, because both consume the same linear fact.

CONSTRUCT 1 · RULES OVER A MULTISET OF FACTS

TOKEN GAME: BRANCH B HAS FIRED



Branch B has fired. The token has been consumed, so branch C cannot fire in this instance.



CONSTRUCT 1 · A RULE FROM THE MODEL

A THROW EVENT AS A REWRITE RULE

```
rule A_Throw_Award_B:
  [ St(pid, 'A', 'throw_award_B'),
    !ModelEdge('A', 'throw_award_B', 'B', m) ]
  --[ Act(pid, 'A', 'A', 'throw_award_B'),
       Sent(pid, 'A', m) ]->
  [ St(pid, 'A', 'catch_order_B'),
    MsgOut(pid, 'A', 'B', m) ]
```

```
rule B_Catch_Award:
  [ St(pid, 'B', 'start'),
    MsgIn(pid, 'A', 'B', m) ]
  --[ Received(pid, 'B', 'A', m) ]->
  [ St(pid, 'B', 'prepare_order') ]
```

- **ST** the token moves from the throw event to the catch event
- **!MODELEDGE** the collaboration graph as data: which message goes on this flow
- **ACT, SENT, RECEIVED** what this step leaves on the trace; lemmas see nothing else
- **MSGOUT, MSGIN** the only way a message leaves or enters a pool

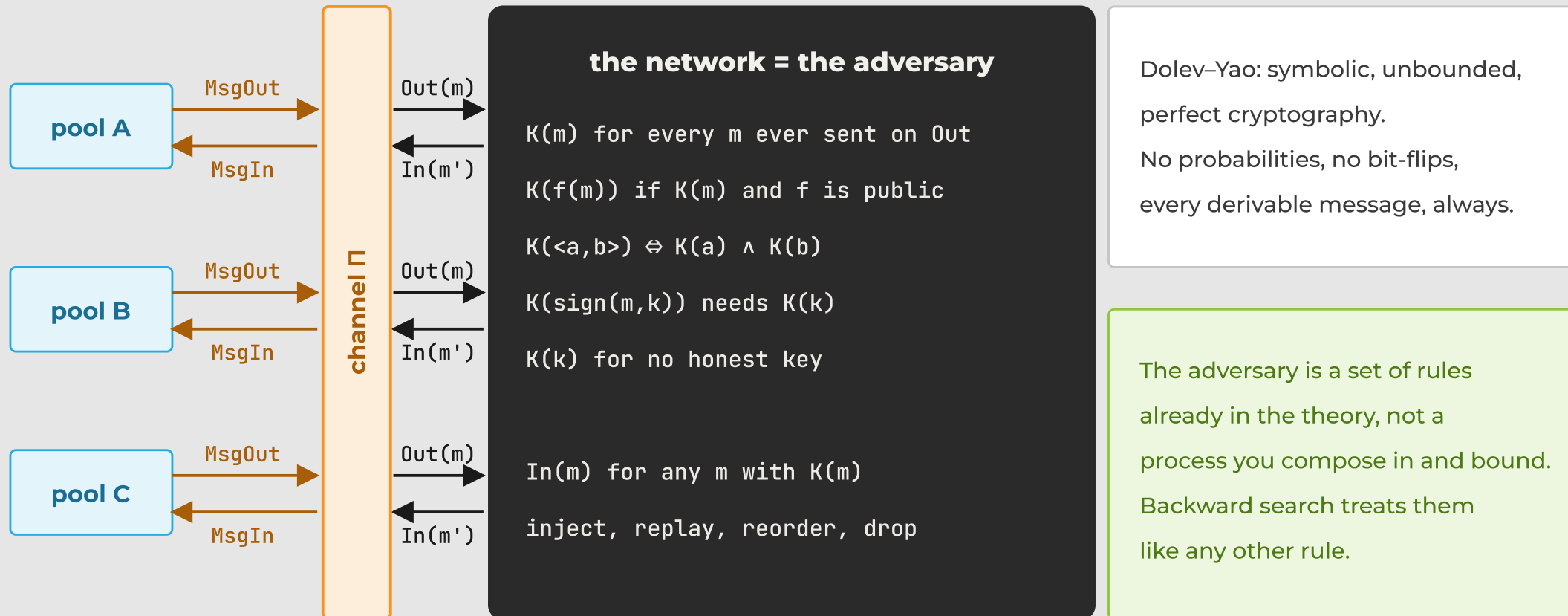
No network. The process layer makes no reference to Out or In.

LINEAR consumed once **!PERSISTENT** never consumed **~FRESH** unguessable new name **\$PUBLIC** known to all
every element of the three pools is encoded by the same pattern



CONSTRUCT 2 · THE DOLEV-YAO ADVERSARY

THE ADVERSARY MODEL



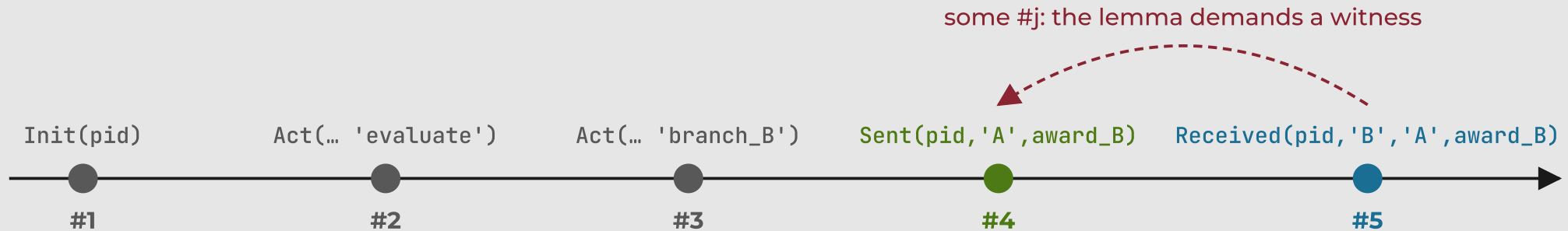
CONSTRUCT 3 · PROPERTIES

LEMMAS ARE TRACE PROPERTIES

lemma non_repudiation:

$\text{All pid pj pi m \#i. Received(pid, pj, pi, m) @ \#i ==> Ex pid_s \#j. Sent(pid_s, pi, m) @ \#j}$

for every trace, for every time point where B accepted m as coming from A, there is a time point where A sent m



Only action facts are visible to a lemma. State facts (St, MsgOut, !Auth ...) are not: choose action facts as your property vocabulary.

CONSTRUCT 3 · PROOF SEARCH

BACKWARD SEARCH: PROOF OR COUNTEREXAMPLE

Backward search. Negate the lemma, ask which rule could have produced this fact, and repeat the question back to the empty state.

- every branch closes in a contradiction: **verified**, for unboundedly many instances
- one branch reaches a consistent trace: **falsified**, with that trace as the witness
- neither: non-termination, the only incompleteness of the method, and where the modeling effort goes: reuse lemmas, restrictions, oracles

```
$ scripts/tamarin --prove -D=PROFILE_LC -D=ADV_A \  
  model/collaboration.spthy  
  
summary of summaries:  
  gateway_exclusive (all-traces): verified (2 steps)  
  no_bifurcation (all-traces): falsified - found trace (9 steps)  
  non_repudiation (all-traces): verified (8 steps)  
  faithful_history (all-traces): falsified - found trace (8 steps)  
  authorized_progression (all-traces): verified (8 steps)  
  instance_isolation (all-traces): falsified - found trace (5 steps)  
  value_secrecy (all-traces): falsified - found trace (4 steps)
```

Every configuration in this tutorial terminates in under five seconds. The lemma names are the model's: no_bifurcation is the one-award property from Part I; the other five are introduced in Part III.



Part III

ONE PROCESS LAYER, SEVERAL CHANNEL LAYERS

ASSUMPTIONS, GUARANTEES, AND CHANNEL LAYERS

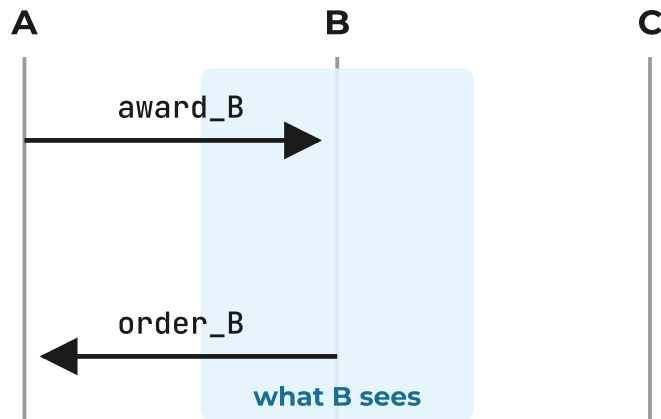
Two facts separate the process layer from the network. Replacing the channel layer replaces one file; the lemmas stay the same.



GUARANTEES · INDISTINGUISHABLE RUNS

TWO RUNS WITH THE SAME OBSERVATIONS FOR B

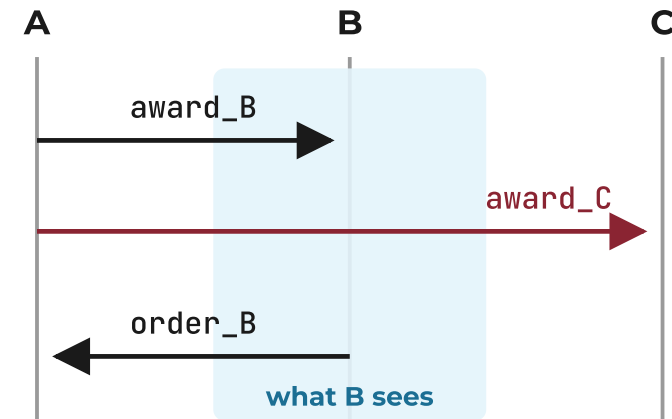
run 1 · A is honest



“I am the sole awardee” is

TRUE here

run 2 · A awards both suppliers



“I am the sole awardee” is

FALSE here

B sees exactly the same in both runs: an award in, an order out.

ASSUMED, GUARANTEED, OR GUARANTEED AT A POINT IN TIME

ASSUMED

The property holds in **some** run consistent with what the participant has seen. Nothing it observed contradicts it.

GUARANTEED

The property holds in **every** run consistent with what the participant has seen. No consistent run violates it.

GUARANTEED AT TIME T

The property held in every consistent run **at the moment the participant checked**. It may have changed since.

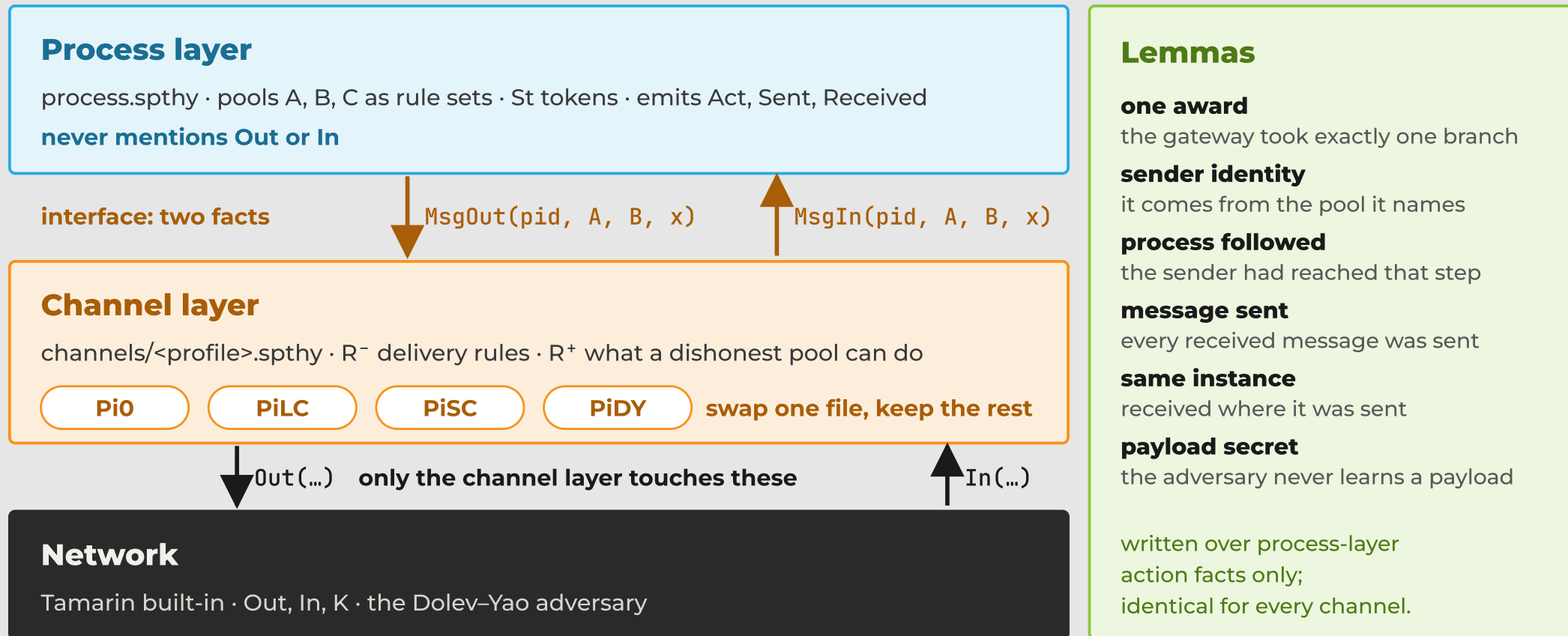
Everything guaranteed is also assumed. The converse holds only when every run is honest. An attacker goal is a run in which a participant's assumption is false. (In the paper: belief $B_i(\varphi)$, knowledge $K_i(\varphi)$, time-stamped knowledge $K_i^t(\varphi)$.)

Runs are Tamarin traces. Two runs look the same to B when B's own action facts are identical in both. BPMN's execution model admits only honest runs, so everything a participant assumes is automatically guaranteed. Admit a dishonest participant and the two differ.



LAYERING · STRUCTURE OF THE THEORY

PROCESS LAYER AND CHANNEL LAYER



Init emits one !Setup(pid) per instance, which is how a channel that does not carry pid still finds one.

THREE CHANNEL LAYERS

PI0 · IDEAL SECURE

```
rule ChanOut_Sec:
  [ MsgOut(pid,$A,$B,x) ]
  --[]->
  [ Sec(pid,$A,$B,x) ]

rule ChanIn_Sec:
  [ Sec(pid,$A,$B,x) ]
  --[]->
  [ MsgIn(pid,$A,$B,x) ]
```

nothing is placed on **Out**, so the adversary learns nothing; linear facts, so no replay; **pid** travels with the message.

PiLC · AUTHENTICATED

```
rule ChanOut_LC:
  [ MsgOut(pid,$A,$B,x) ]
  --[]->
  [ !Auth($A,$B,x), Out(x) ]

rule ChanIn_LC:
  [ !Auth($A,$B,x), !Setup(pid) ]
  --[]->
  [ MsgIn(pid,$A,$B,x) ]
```

sender bound, payload leaked, persistent facts permit replay, **pid** taken from any instance.

PiDY · RAW NETWORK

```
rule ChanOut_DY:
  [ MsgOut(pid,$A,$B,x) ]
  --[]->
  [ Out(<$A,$B,x>) ]

rule ChanIn_DY:
  [ In(<s,r,x>), !Setup(pid) ]
  --[]->
  [ MsgIn(pid,s,r,x) ]
```

names come from the wire, unchecked. The adversary can act as any participant.

what the channel file does	Pi0	PiLC	PiDY
puts the message on the wire (Out)	no	payload only	yes
delivers from the wire (In)	no	no	yes
channel fact	linear	persistent	none
instance id travels with the message	yes	no	no



LAYERING · WHAT A DISHONEST POOL CAN DO

THE DISHONEST POOL

```
// process.spthy
#ifdef ADV_A
rule Adv_Mark_A:  [] --[]-> [ !Adv('A') ]
#else
  ... the honest rules of pool A ...
#endif
```

```
// channels/piLC.spthy (R+ for this platform)
rule Adv_Send_LC:
  [ !Adv($A),
    !ModelEdge($A, e, $B, m),
    Fr(~pid) ]
  --[ Sent(~pid, $A, m) ]->
  [ MsgOut(~pid, $A, $B, m) ]
```

Attacker configuration $A = (P_{adv}, \Pi)$, $\Pi = (R^+, R^-)$

- the dishonest pool's honest rules are gone; one marker fact remains
- **R⁺**: what that pool can do *on this platform*. Here: send any model-valid message under its own name, **without consuming a token**
- **R⁻**: the delivery rules, enforced regardless of honesty
- collusion needs no extra rules: `!Adv($A)` matches any marked pool

RESULTS ACROSS PLATFORMS

channel · dishonest pool	one award	sender identity	process followed	message sent	same instance	payload secret
Pi0 · none	✓	✓	✓	✓	✓	✓
PiLC · A	✗	✓	✗	✓	✗	✗
PiLC · B	✓	✓	✗	✓	✗	✗
PiSC · A	✓	✓	✓	✓	✓	✗
PiDY · A	✗	✗	✗	✗	✗	✗

Pi0 and PiDY are diagnostics: the encoding is right (all hold) and platform constraints are necessary (all fail). PiSC is smart-contract orchestration: a contract validates every transition against the graph and holds one linear state token per instance. Six safety properties; liveness (the others will continue) is out of scope, since no platform can force a step.

The contract gives every safety property and removes confidentiality. The authenticated channel preserves identity and nothing else. Reproduce it with `scripts/verify.sh` in under a minute. The full matrix, four pools and five platforms, is in the paper.



LAYERING · THE MATRIX, READ PER PLATFORM

WHAT EACH PLATFORM GUARANTEES

platform	R+ the dishonest pool may	R- the platform enforces	assumed or guaranteed?
PiLC loose coupling	send any model-valid message under its own name, independently to each recipient	sender identity on the channel	identity: guaranteed . everything else: assumed
PiSC smart contract	submit any valid transition; read all state	contract validates against the graph; linear state = atomicity	every safety property: guaranteed . confidentiality: lost
PiVC credentials (paper only)	present its own credentials; choose timing	non-invocation tokens, revocation in a registry	identity: guaranteed. gateway: guaranteed at check time , or only assumed, depending on registry atomicity

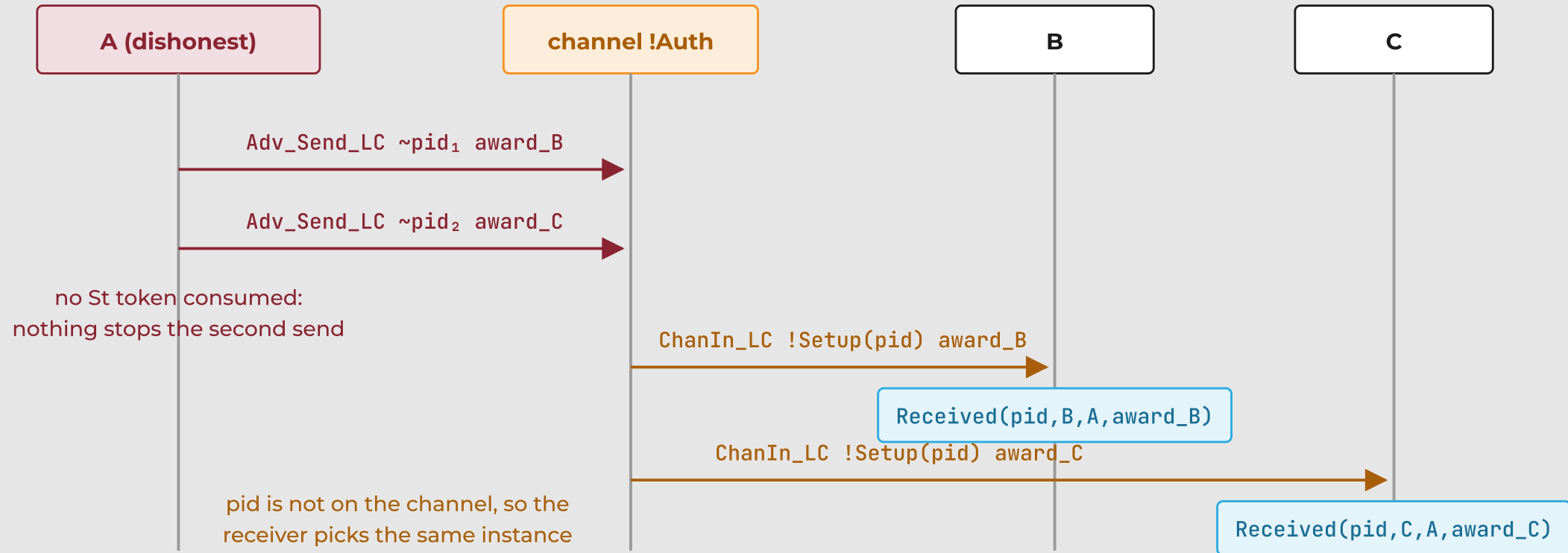
The one-conjunct result (PiVC, in the paper): the bifurcation check under an atomic registry and under a registry with propagation delay differ by a single temporal guard.

```
CheckAlt(n) @ #i ==> not(Ex #k. Revoked(n) @ #k)
CheckAlt(n) @ #i ==> not(Ex #k. Revoked(n) @ #k & #k < #i)
```

Add `& #k < #i` and one award flips from ✓ to ✕: a TOCTOU window.
Guaranteed at check time, only assumed by commit time.



THE COUNTEREXAMPLE FOR ONE AWARD ON PILC



The trace Tamarin returns, redrawn: two `Adv_Send_LC` firings without a token, and a channel that does not carry `pid`, so both awards land in the same instance. No cryptographic weakness is involved.

A TIMING WINDOW UNDER NON-ATOMIC REVOCATION

```
CheckAlt(n) @ #i ==> not(Ex #k. Revoked(n) @ #k)
CheckAlt(n) @ #i ==> not(Ex #k. Revoked(n) @ #k & #k < #i)
```

Atomic registry. The first restriction: a revoked credential is never accepted, wherever the revocation sits on the trace. The one-award guarantee holds.

Registry with propagation delay. With `& #k < #i` only revocations that precede the check count. One award fails: the guarantee held at check time and no longer holds at commit time.

PiVC · dishonest A	one award	sender identity	message sent
atomic registry	✓	✓	✓
registry with propagation delay	✗	✓	✓

The credential platform (PiVC) is in the paper and the artifact, not in today's model. Its full row is in the backup.



Part IV

CHECKING THE IDEALIZATION

THE REALIZED CHANNEL AGAINST THE IDEAL ONE

The realized channel is checked against the same lemmas. Equal results are evidence for the idealization, not a refinement proof.



THE AUTHENTICATED CHANNEL, IDEAL AND REALIZED

```
// piLC.spthy: ideal
rule ChanOut_LC:
  [ MsgOut(pid,$A,$B,x) ]
  --[]->
  [ !Auth($A,$B,x), Out(x) ]

rule ChanIn_LC:
  [ !Auth($A,$B,x), !Setup(pid) ]
  --[]->
  [ MsgIn(pid,$A,$B,x) ]
```

```
// piLCcrypto.spthy: realized
rule Register_Key:
  [ Fr(~ltk) ] --[ KeyRegistered($P) ]->
  [ !Ltk($P,~ltk), !Pk($P,pk(~ltk)), Out(pk(~ltk)) ]

rule ChanOut_LC_Crypto:
  [ MsgOut(pid,$A,$B,x), !Ltk($A,k) ]
  --[]->
  [ Out(<$A,$B,x, sign(<$A,$B,x>,k)>) ]

rule ChanIn_LC_Crypto:
  [ In(<$A,$B,x,s>), !Pk($A,pkA), !Setup(pid) ]
  --[ Eq(verify(s,<$A,$B,x>,pkA), true) ]->
  [ MsgIn(pid,$A,$B,x) ]

restriction Eq: "All a b #i. Eq(a,b) @ #i ==> a = b"
```

The equational theory does the work: $\text{verify}(\text{sign}(m, k), m, \text{pk}(k)) = \text{true}$, and nothing else verifies. The dishonest pool signs with its own key; that is its R^+ capability on this platform.



WHAT EQUAL RESULTS PROVE

channel · dishonest A	one award	sender identity	process followed	message sent	same instance	payload secret
PiLC · ideal	×	✓	×	✓	×	×
PiLC · realized with signatures	×	✓	×	✓	×	×

Evidence that the ideal layer was faithful *for these properties*: the realization satisfies the same lemmas as the idealization and fails the same ones. The check is automatic and detects idealizations that are too strong.

Tamarin has no notion of "realized $\preceq$ ideal", and equal results are not trace inclusion. A refinement argument, with an interface contract stated as lemmas, proved once below and assumed above, is a proof obligation outside the tool and the subject of our current work.

Related: secure pseudonymous channels (Mödersheim, Viganò), stateful protocol composition (Hess, Mödersheim, Brucker), and the Tamarin manual's channel models.



Part V

EXERCISE

A CHANNEL LAYER TO WRITE

Warm up with the channel of the lecture,
then build one the lecture does not have.



EXERCISE · A CHANNEL OF YOUR OWN

THE EXERCISE

```
// exercise/mychannel.spthy
rule ChanOut_Mine:
  [ MsgOut(pid, $A, $B, x) ]
  --[]->
  [ ??? ]

rule ChanIn_Mine:
  [ ??? ]
  --[]->
  [ MsgIn(pid, $A, $B, x) ]

rule Adv_Send_Mine: // given
  [ !Adv($A), !ModelEdge($A,e,$B,m), Fr(~pid) ]
  --[ Sent(~pid,$A,m) ]->
  [ MsgOut(~pid,$A,$B,m) ]
```

Warm-up (2 min): type PiLC back in, predict its column, run `scripts/check-exercise.sh --warmup`.

Task (12 min): build PiQ, a private queue. Sender identity unforgeable · payload never on the wire · each message handed over at most once · `pid` still not carried.

The exercise adds an eighth lemma, `no_duplication`. The file gives the target column. Two cells of the warm-up column move and six stay: name the two before you edit, and say why four failures survive.

VACUITY

The skeleton as shipped delivers nothing, so every safety lemma holds **vacuously**; the sanity lemma `delivery_works` detects this. Stretch: realize the warm-up channel with signatures and re-check.



THE REPOSITORY

```
model/
  collaboration.spthy    entry + lemmas
  process.spthy          the three pools
  channels/pi0.spthy
  channels/piLC.spthy
  channels/piLCcrypto.spthy
  channels/piSC.spthy
  channels/piDY.spthy
exercise/
  exercise.spthy
  mychannel.spthy        ← you
scripts/
  tamarin                local or Docker
  verify.sh              the matrix
  check-exercise.sh
slides/                  this deck, Marp
```

QUICK START

```
git clone https://github.com/BlackLight54/avn-tamarin-tutorial
cd avn-tamarin-tutorial && scripts/verify.sh
```

Needs Docker, or a local `tamarin-prover` ≥ 1.8 with Maude.

REFERENCES

Farkas, Kocsis. *Trust No Pool: Formal Security Analysis of BPMN Collaborations*. BPM 2026, foundations track, to appear. Artifact: doi.org/10.5281/zenodo.20273971.



this repository
model, exercise, deck



Tamarin team tutorials
exercises included, CC-BY



the Tamarin book
tamarin-prover.com/book



OPEN PROBLEMS

A certified BPMN-to-Tamarin translation. The encoding is currently a manual pattern. A correspondence specification and a certified translator, in Lean for instance, would turn the argument of Part II into a theorem.

Lemmas as contracts across a layered stack. Consensus, credential, and orchestration layers, each with its own lemmas, proved once and used by the layer above without re-proving its internals. Tamarin offers no composition theorem for this.

Tamarin as an error-propagation analyzer. Petri nets support EPA; Petri nets are multiset rewriting; so Tamarin's backward search could look for traces that cause errors rather than traces that exploit them. No such tool exists yet.

MARTIN FARKAS · IMRE KOCSIS

martin.farkas@edu.bme.hu · kocsis.imre@vik.bme.hu · ftsrg.mit.bme.hu · thanks to Levente Bajczi and Zsófia Ádám for the slot



BACKUP

—
THE PLATFORM TABLE · A TERMINAL RUN

LAYERING · A LIVE RUN OF THE MATRIX

RUNNING THE MATRIX

```
$ git clone https://github.com/BlackLight54/avn-tamarin-tutorial
$ cd avn-tamarin-tutorial
$ scripts/verify.sh
Running 6 configuration(s) with 8 parallel job(s)...
  done: Pi0 (3 s)
  done: PiLC_A (2 s)
  ...
config      G1    G2    G3    G4    G7    G6a    expected
Pi0          V     V     V     V     V     V      ok
PiLC_A       F     V     F     V     F     F      ok
PiLC_B       V     V     F     V     F     F      ok
PiLCcrypto_A F     V     F     V     F     F      ok
PiSC_A       V     V     V     V     V     F      ok
PiDY_A       F     F     F     F     F     F      ok
All results match the expected matrix.
```

- `scripts/tamarin` runs the prover locally if installed, otherwise from a pinned Docker image
- `scripts/verify.sh` is the whole matrix with expected values inline, so it doubles as a regression test
- `scripts/tamarin-gui` for the GUI on `localhost:3001`: proof trees and attack graphs



BACKUP · QUESTIONS

PREPARED ANSWERS, ONE OF TWO

question	answer
Every lemma is all-traces, but you called security existential.	Each lemma is an attacker goal negated, so falsified means the attack trace exists. <code>collaboration.spthy</code> says so above each lemma.
What stops the adversary from simply being the network under PiLC?	<code>piLC.spthy</code> puts only the payload on <code>out</code> and never consumes <code>In</code> , and <code>!Auth</code> comes from <code>chanOut_LC</code> alone. Identity holds, confidentiality does not.
Are the results sensitive to the number of instances?	No. <code>Init</code> is unrestricted, and verified in Tamarin holds for unboundedly many instances.
Is <code>no_bifurcation</code> about the gateway or about message equality?	Two <code>Received</code> facts in one <code>pid</code> whose payloads are flagged <code>Exclusive</code> must be equal. <code>ModelSetup</code> flags the award pair and the order pair.
Why does PiLC · B keep one award while PiLC · A loses it?	Only pool A owns the gateway, so a dishonest B has no exclusive branch to violate. <code>results/PiLC_B.log</code> : verified in 40 steps.
Equal ideal and realized results: is that not the same lemmas twice?	It is, and that is the claim: it detects an idealization that is too strong. It is not trace inclusion, and there is no refinement theorem.



BACKUP · QUESTIONS

PREPARED ANSWERS, TWO OF TWO

question	answer
Does faithful_history fail even for an honest pool under PiLC?	It fails for a dishonest A and for a dishonest B, and holds on PiO and PiSC. The authenticated channel does not witness process position.
Why is liveness absent?	No platform can force a participant to take a step, so all six properties are safety properties.
How do you know the encoding is not vacuous?	PiO runs with <code>-D=SANITY</code> , which adds <code>lemma executable: exists-trace</code> , verified in 5 steps. The exercise ships the same trap as <code>delivery_works</code> .
Is the BPMN-to-Tamarin encoding proved correct?	No. It is a manual pattern, and a certified translation is the first open problem on slide 36.
Could a dishonest pool leak another pool's key under PiLCcrypto?	No. <code>piLCcrypto.spthy</code> has no key-reveal rule: only <code>pk(~ltk)</code> reaches <code>out</code> , never <code>!Ltk</code> .

